새싹 학습 노트 · Day13~16 · 2026.11.04~11.09 · Vision·멀티모달 AI 응용 1~4/4 · 4일 종합

이미지 전처리에서 분류·객체 탐지·OCR까지

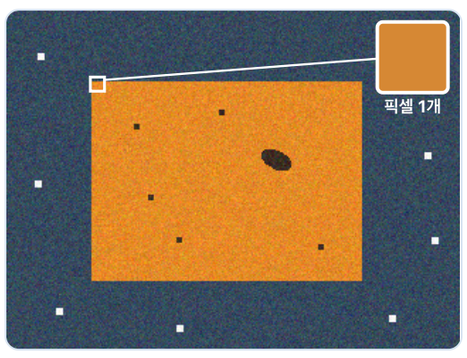
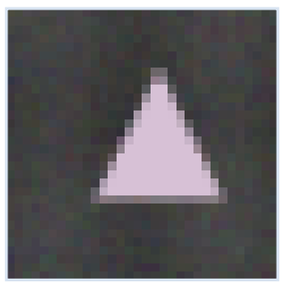
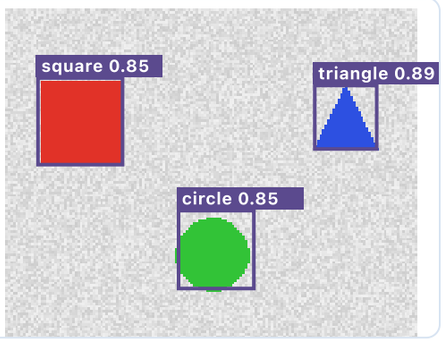
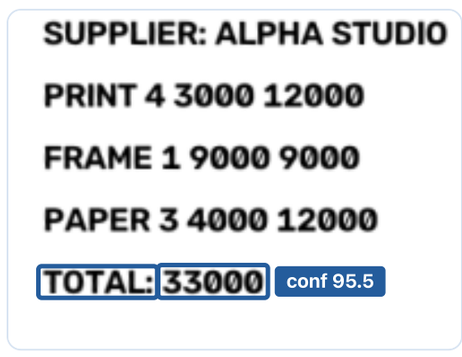
4일 요약 Day13에서 이미지를 숫자 배열로 다루는 법을 배우고, 이를 바탕으로 Day14 **이미지 분류**, Day15 **객체 탐지·추적**, Day16 **텍스트 인식(OCR)·VLM**으로 넘어갔습니다. 날이 갈수록 모델 구조에 단계가 추가되고, 사람이 정하는 파라미터도 전처리 값에서 학습 설정, 결과 필터링 기준으로 바뀝니다.

매일 수업 순서 데이터 준비 → 모델·처리 → 평가 → 틀린 사례 분석 → 통합·재실행 (7·8교시)

1 4일 흐름

날짜별 주제, 질문, 실행 결과

	Day13 · 11/04	Day14 · 11/05	Day15 · 11/06	Day16 · 11/09
	이미지 다루기 (전처리)	이미지 분류	객체 탐지·추적	텍스트 인식 (OCR)·VLM
	OpenCV	PyTorch · torchvision	Ultralytics YOLO	Tesseract · Transformers
질문	이미지는 컴퓨터에서 **어떻게 표현**되나?	이 이미지는 **어떤 클래스**인가?	**어떤 객체가 어디에** 있나?	이미지에 **어떤 글자**가 있나?

결과 예시
각 날 실습 데이터로 실행한 결과
	
	`img.shape  # (높이, 너비, 채널)`
	`(240, 320, 3)`
	`img[52, 64]  # 픽셀 1개 = B, G, R`
	`[52, 136, 214]`
	이미지는 숫자 배열이고 픽셀마다 0~255 값 (Day13 합성 제품 사진)
		
		`# 클래스 점수 3개 중 가장 큰 것`
		`"triangle"`
		`# circle · square · triangle`
		이미지 한 장에 클래스 하나 (Day14 기본 데이터)
			
			`# 객체마다 클래스 · 점수 · Box`
			`triangle 0.89 [120,30,144,55]`
			`circle  0.85 [67,79,97,109]`
			`square  0.85 [13,27,46,61]`
			객체마다 클래스·점수·Box (Day15 응용 val, conf 0.5)
				
				`{"raw": "33000", "value": 33000,`
				`"conf": 95.5,`
				`"box": [146, 369, 89, 21]}`
				단어의 글자·위치를 JSON 필드로 정리 (Day16 청구서, PSM 6)

전날과 달라진 점		**13 → 14 필터 값을 모델이 학습** Day13의 Blur Kernel과 CNN의 합성곱은 같은 계산입니다. Day13에서는 Kernel 값을 사람이 정했고, Day14에서는 Backbone(특징 추출기)이 데이터로 학습합니다. 사전학습된 Backbone을 가져와 Head(분류기)만 새로 학습하는 방식이 전이학습입니다. Day13 개념지도 · Day14 p.5	**14 → 15 클래스에 위치(Box) 추가** Head가 클래스와 함께 **Box와 Confidence**를 출력합니다. 후보가 수백 개 나오기 때문에 **conf와 NMS**로 걸러 내고, 영상에서는 **Tracker**가 Frame 사이의 같은 객체에 같은 ID를 붙입니다. Day15 p.4·9	**15 → 16 Box 안의 글자 인식** 글자 영역을 Box로 찾고(Day15와 같은 형식) 그 안의 문자를 인식해 문자열을 얻은 뒤, 필요한 값을 JSON 필드로 정리합니다. VLM은 언어모델이 답을 문장으로 만듭니다. 문서에 없는 값을 만들어 낼 수 있어 OCR 원문과 비교해 확인합니다. Day15 p.104 · Day16 p.4·7

2 날짜별 비교

열은 위와 같은 날짜입니다. 그날 새로 추가된 부분은 날짜 색으로 표시했습니다

내부 구조

입력에서 결과까지의 처리 순서

	Day13 · 11/04	Day14 · 11/05	Day15 · 11/06	Day16 · 11/09
	이미지	이미지	이미지	OCR
처리 순서	이미지 → Gray → Resize → Blur → Threshold → Morphology → **규칙 기반 판단**	이미지 → 입력 변환 → **Backbone (빌려 옴)** → **Head (새로 학습)** → 클래스 점수	이미지 → Backbone → **Neck** → Head (클래스·**Box**·점수) → **conf → NMS** → **Tracker**	전처리 → 글자 영역 탐지 → **문자 인식** → **필드 JSON** VLM 이미지 + 질문 → Vision Encoder → **언어모델** → 답
	모델 없음. 파라미터는 사람이 지정	학습하는 모델이 처음 등장	객체마다 결과 1개	여러 모델과 단계를 연결

메소드

단계별 대표 메소드 — **Day15부터 학습·추론은 함수 한 줄(model.train, pipeline)이고, 결과를 확인하는 코드가 늘어난다**

	OpenCV	PyTorch · torchvision	Ultralytics · torchvision.ops	Tesseract · Transformers
준비	`cv2.imread` `cvtColor` `resize`	`ImageFolder` `transforms.Normalize`	`box_convert` 라벨 좌표 변환	`adaptiveThreshold` `warpPerspective`
처리·모델	`GaussianBlur` `Canny` `threshold` `morphologyEx`	`requires_grad_(False)` `nn.Linear` `CrossEntropyLoss` `optimizer.step`	`YOLO()` `model.train` `model.predict` `model.track`	`image_to_data` `pipeline("image-text-to-text")`
결과 확인	`imwrite` `VideoWriter`	`argmax` `f1_score` `confusion_matrix`	`result.boxes` `box_iou` `model.val`	`re.search` `jiwer.cer` `json.dump`

파라미터

날짜별로 사람이 정하는 주요 값

주요 파라미터

전처리 설정

`ksize`	Blur에서 평균을 낼 주변 영역 크기
`thresh`	흰색(물체)으로 나눌 밝기 기준값
`interpolation`	Resize할 때 새 픽셀 값을 계산하는 방법
`fps`	저장할 영상의 초당 프레임 수

학습 설정

`requires_grad`	학습할 층 선택 (Head-only · Partial · Full)
`lr`	한 번 업데이트할 때 가중치를 바꾸는 크기
`epochs`	학습 데이터 전체를 반복하는 횟수
`weight`	클래스 불균형을 보정하는 클래스별 가중치

결과 필터링·평가 기준

`conf`	결과로 남길 최소 점수
`iou` (NMS)	중복 Box로 보고 제거할 겹침 기준
평가 `IoU`	정답으로 인정할 겹침 기준 (0.5 / 0.5~0.95)
`imgsz`	모델 입력 이미지 크기

OCR 입력 조건 · VLM 생성 설정

`blockSize·C`	Adaptive Threshold의 주변 영역 크기와 보정값
`--psm`	Tesseract의 페이지 구조 가정 (6 텍스트 블록, 7 한 줄, 8 한 단어)
`max_new_tokens`	VLM이 생성할 최대 토큰 수
`do_sample`	샘플링 여부 (False면 같은 입력에 같은 답)

평가 기준

결과를 무엇과 비교해 평가하나 — **출력이 복잡해질수록 따로 확인할 지표가 늘어난다**

	Day13 · 11/04	Day14 · 11/05	Day15 · 11/06	Day16 · 11/09
지표	처리 결과의 형식 확인 **shape·dtype** **재읽기**	전체 성능과 클래스별 성능 확인 **Accuracy** ≠ **Macro F1** · Confusion matrix	클래스·위치·중복을 함께 확인 **클래스** + **위치 IoU** + **중복 아님** → mAP50·mAP50-95	글자·필드·근거를 따로 확인 **CER** ≠ **필드 정답** ≠ **근거 일치**

출처

강의: Vision·멀티모달 AI 응용 Day13~16 강의 PDF (쪽 표기는 각 날 `DayNN_강의구조_핵심설명.md` 기준).

그림: Day13 개념지도(`make_images.py`, OpenCV)와 Day14~16 oneday 요약(`facts.json`)의 실제 실행 결과를 다시 썼습니다. Day14는 PyTorch, Day15는 Ultralytics YOLO26n, Day16은 Tesseract 5.5.2입니다. 실행 환경은 각 날 `DESIGN.md`에 있습니다.

메소드는 각 날 oneday 요약의 핵심 코드(Day13은 강의 p.85 핵심 함수 체크리스트와 개념지도)에서 단계마다 대표만 골랐습니다.